53. Docker With vs Without Kubernetes

1. One-line answer

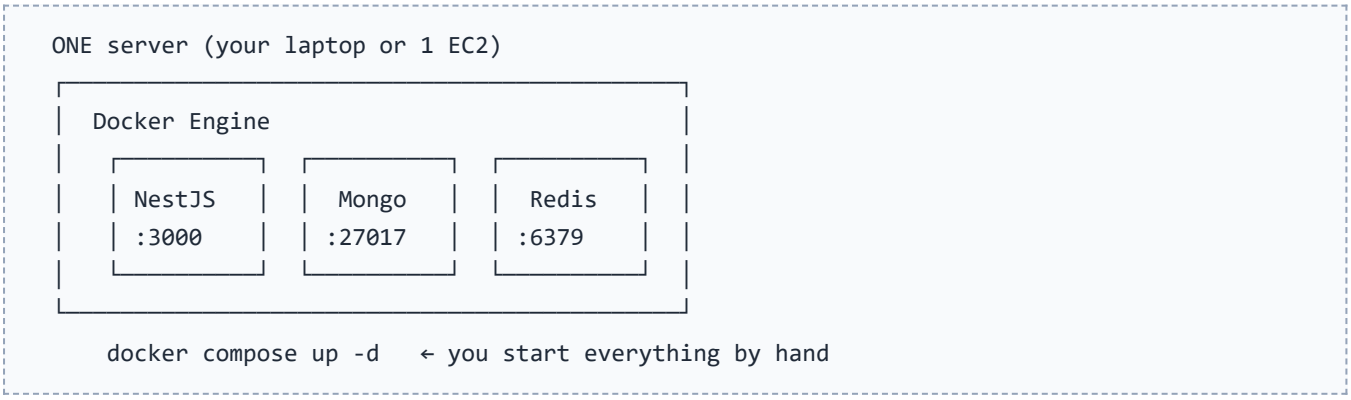
Tool	Job	Real-life example
Docker	**Builds and runs** containers (the boxes)	A **shipping container** 🎁 (and the crane that loads one)
Kubernetes (K8s)	**Manages many containers across many servers**: it starts them, restarts them, scales them and connects them	The **port manager / control tower** 🏗️ that decides which ship carries which container, replaces lost ones, and adds more when demand grows

They are not competitors. Docker **makes** the container image. Kubernetes **runs and looks after** lots of those containers. Usually you use **both**: Docker to build, Kubernetes to run in production.

Restaurant example 🍽️
Docker = a **cook** who can make a dish in a standard tiffin box.
Docker alone = a small dhaba with 1 cook. If the cook is sick, the shop closes. If 500 customers come, the cook can't keep up.
Kubernetes = the **restaurant manager** of a big chain. They keep "always 5 cooks working", replace a sick cook straight away, call in extra cooks at lunch rush, and send customers to whichever counter is free.

2. Docker WITHOUT Kubernetes

This is what you've done so far: `docker run` or `docker compose up` on **one machine**.



✅ This is perfect for:

- Local development
- Learning, small projects, college projects, demos
- Small apps with low traffic on 1 server
- When simplicity matters more than high availability

❌ Problems when the app grows

Situation	With Docker alone...
The NestJS container crashes at 3 AM	`restart: always` helps, but if the **whole server** dies, everything is down until you fix it 😔
Traffic jumps 10×	You must SSH in and start more containers by hand, and set up your own load balancing
You need 5 servers	Compose works on **one** machine. Who decides which container goes on which server?
Deploying a new version	Stop old, start new, so there's **downtime**. If the new version is buggy, you roll back by hand.
Containers on different servers need to talk	You manage IPs and networking yourself
Secrets and config for 20 services	`.env` files copied everywhere

As apps grew to **hundreds of containers on dozens of servers** (microservices), managing them by hand became impossible. That's why **container orchestrators** were created, and Kubernetes won.

3. What is Kubernetes?

Kubernetes is an open-source system that automatically deploys, scales, heals and connects containers across a group of servers (a cluster).

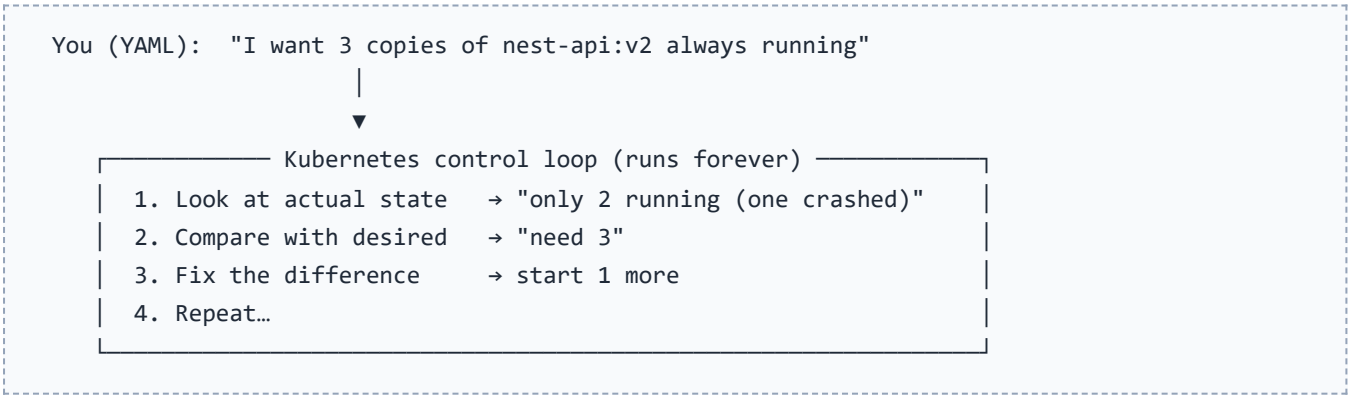
History (why it was introduced)

- Google ran **billions of containers a week** using an internal system called **Borg** (from the mid-2000s).
- In **2014**, Google engineers (Joe Beda, Brendan Burns, Craig McLuckie and others) built **Kubernetes**, an open-source system inspired by Borg, so everyone could run containers like Google does.
- **2015:** v1.0 released and donated to the new **CNCF** (Cloud Native Computing Foundation), so no single company owns it.
- It beat rivals like Docker Swarm and Apache Mesos, and became the **industry standard**. Every cloud offers it: **EKS** (AWS), **AKS** (Azure), **GKE** (Google).

> **The name:** *Kubernetes* is Greek for **helmsman** (the person who steers a ship). That's why the logo is a ship's wheel ☸. **K8s** = K + 8 letters + s.

The big idea: desired state

With Docker you give **commands**: "start this container". With Kubernetes you **describe what you want**, and it keeps making reality match.

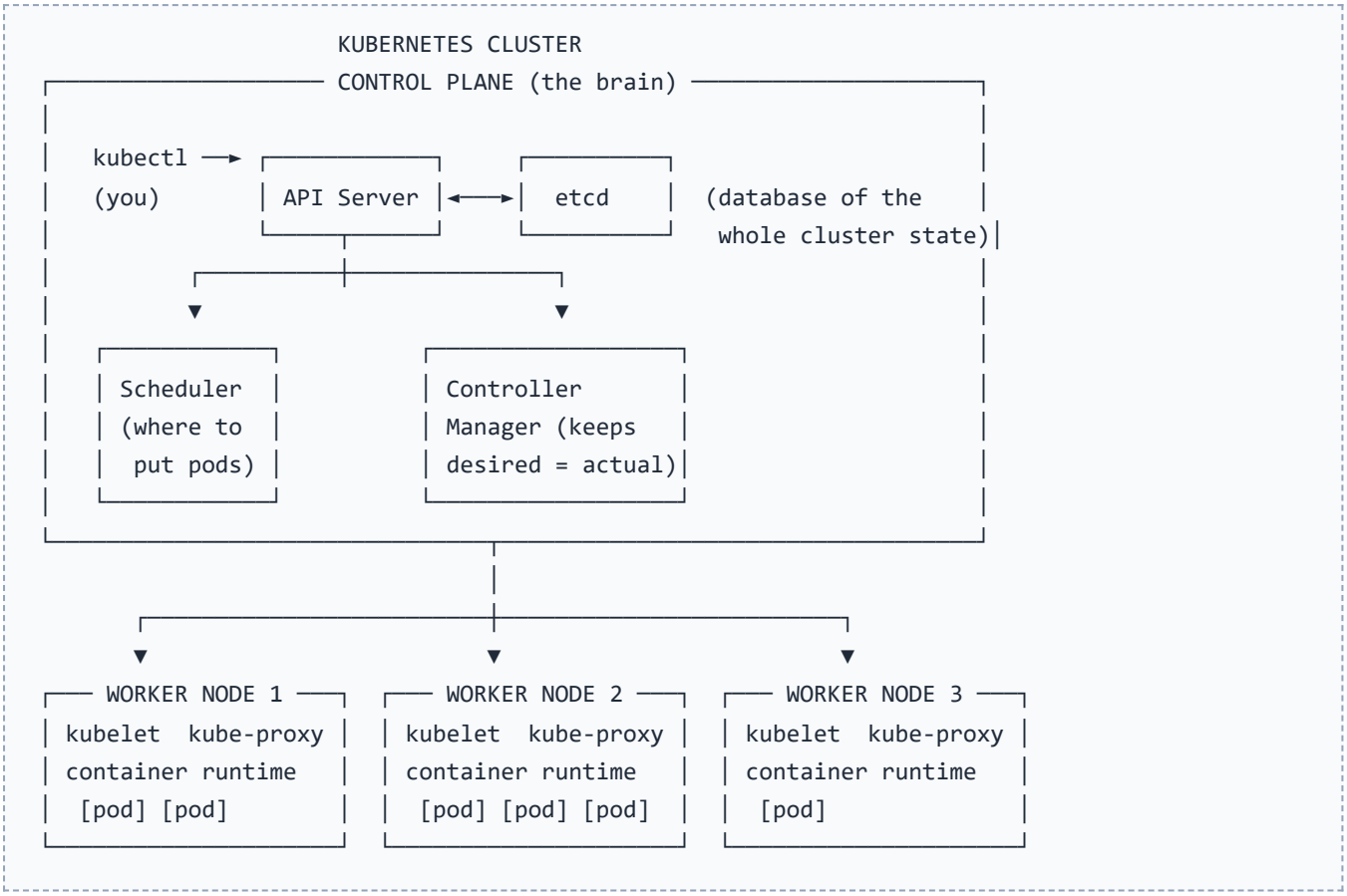


> **AC thermostat example ❄:** You set 24°C (desired state). You don't switch the compressor on and off yourself; the AC keeps checking and adjusting. Kubernetes is a thermostat for containers.

4. What Kubernetes gives you

Feature	Meaning
Self-healing	A container crashes, so it's restarted. A whole server dies, so its containers are moved to healthy servers.
Scaling	`replicas: 3 → 10` with one command, or **automatically** based on CPU (HPA)
Load balancing	A **Service** spreads traffic across all copies
Rolling updates	Replaces old pods with new ones a few at a time, so there's **zero downtime**
Rollback	New version broken? Run `kubectl rollout undo`
Service discovery	Pods find each other by name (`mongo`), not by IP
Config and secrets	ConfigMaps and Secrets are stored centrally and injected as env vars
Scheduling	Decides which server has enough CPU/RAM for each container
Storage	Attaches persistent disks (PVCs) to databases, even if they move servers

5. How Kubernetes works: architecture



Part	Job	Example
Cluster	The whole group of machines	The whole restaurant chain
Control plane	The brain that makes decisions	Head office
API Server	The front door. Everything (kubectl, nodes) talks through it.	Reception desk
etcd	A key-value DB storing the entire cluster state	Head office register
Scheduler	Picks which node runs each new pod	Shift planner
Controller Manager	Runs the loops that fix desired vs actual state	Supervisor doing rounds
Node	A worker machine (VM or physical) that runs pods	One branch kitchen
kubelet	An agent on each node that starts and stops containers as told	Branch manager
kube-proxy	Handles networking rules so Services reach pods	The branch's phone line
Container runtime	Actually runs the containers (containerd, CRI-O)	The cook

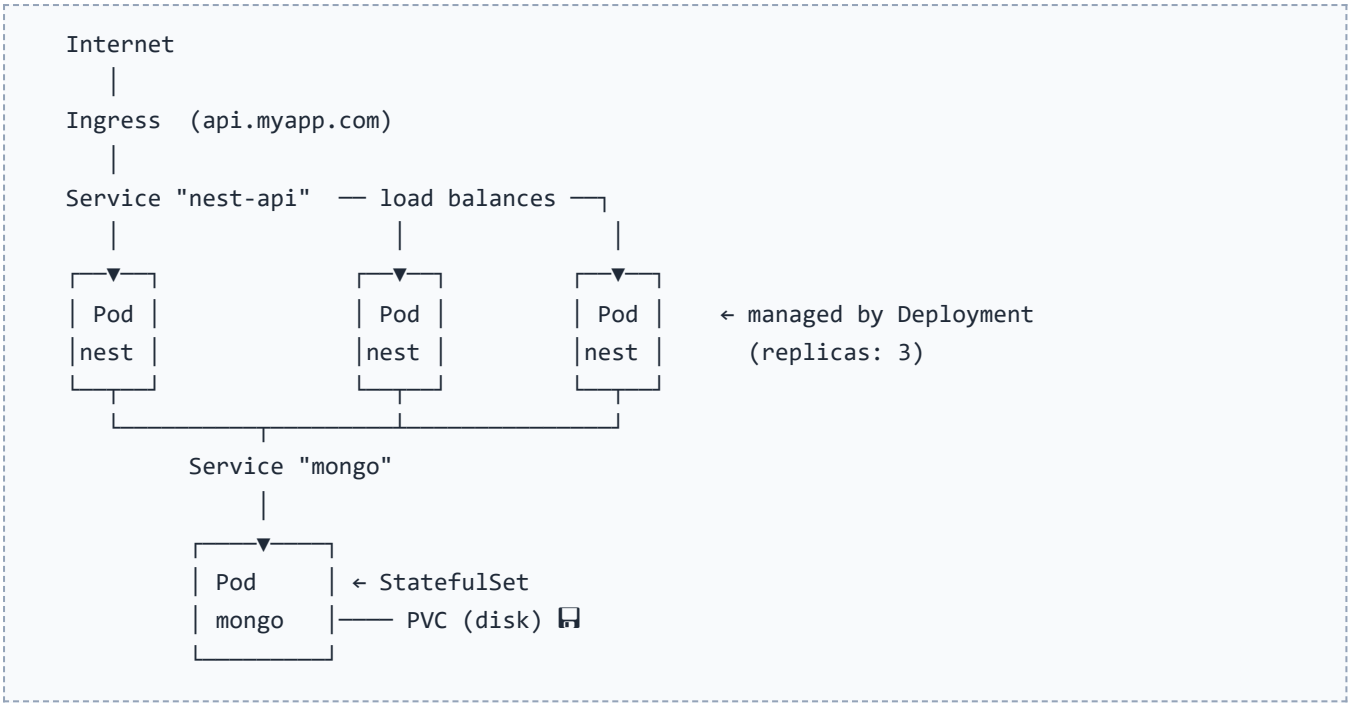
> **"Does Kubernetes still use Docker?"** Kubernetes stopped talking to the Docker Engine directly in v1.24 (2022). It now uses **containerd**, the runtime that is also inside Docker. But **images built with Docker work perfectly**, because they follow the same OCI standard. You still use `docker build` + `docker push`, and K8s runs those images.

What happens when you deploy (step by step)

1. You run `kubectl apply -f deployment.yaml` ("3 replicas of nest-api").
2. The **API Server** checks it and saves it in **etcd**.
3. The **Controller Manager** sees "want 3, have 0" and creates 3 pod definitions.
4. The **Scheduler** picks a node for each pod (based on free CPU/RAM).
5. The **kubelet** on each chosen node pulls the image and starts the container.
6. A pod dies later? The controller notices "have 2" and creates a new one. Steps 4–5 repeat. ♻️

6. Kubernetes objects (the vocabulary)

Object	Simple meaning	Docker / Compose equivalent
Pod	The smallest unit: 1 (or a few tightly linked) containers sharing a network and storage	~ one container
ReplicaSet	Keeps N copies of a pod running	~ `deploy.replicas`
Deployment	Manages ReplicaSets, rolling updates and rollbacks. **You use this most.**	~ a service in compose
Service	A stable name/IP + load balancer in front of pods (pods come and go, their IPs change)	~ compose service name + ports
Ingress	HTTP routing from outside: `api.myapp.com → nest service`	~ Nginx reverse proxy
ConfigMap	Non-secret config (env vars, files)	~ `environment:`
Secret	Passwords and keys (base64-encoded; enable encryption at rest for real safety)	~ `.env`
PersistentVolumeClaim (PVC)	A request for a disk that survives pod restarts	~ named volume
StatefulSet	Like a Deployment, but for databases (stable names + own disk per pod)	~ DB service + volume
Namespace	A virtual folder to separate environments or teams (dev, prod)	~ separate compose projects
HPA	Horizontal Pod Autoscaler: adds or removes pods based on CPU or other metrics	❌ none in Compose


Internet
Ingress (api.myapp.com)
Service "nest-api" ── load balances ──┐
Pod nest
Pod nest
Pod nest
← managed by Deployment
(replicas: 3)
Service "mongo"
Pod mongo
← StatefulSet
PVC (disk)

7. Same app, both ways: NestJS + MongoDB

A) WITHOUT Kubernetes: docker-compose.yml

```
services:
  api:
    image: gourish/nest-api:v1
    ports: ["3000:3000"]
    environment:
      MONGO_URI: mongodb://mongo:27017/schooldb
    restart: always
  mongo:
    image: mongo:7
    volumes: [mongo-data:/data/db]
volumes:
  mongo-data:
```

```
docker compose up -d       # one machine, one copy of each
```

B) WITH Kubernetes: k8s/api.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nest-api
spec:
  replicas: 3                          # ← 3 copies, kept alive forever
  selector:
    matchLabels: { app: nest-api }
  template:
    metadata:
      labels: { app: nest-api }
    spec:
      containers:
        - name: nest-api
          image: gourish/nest-api:v1   # ← the SAME Docker image
          ports:
            - containerPort: 3000
          env:
            - name: MONGO_URI
              valueFrom:
                secretKeyRef: { name: app-secrets, key: MONGO_URI }
          resources:
            requests: { cpu: "100m", memory: "128Mi" }
            limits:   { cpu: "500m", memory: "256Mi" }
          readinessProbe:              # send traffic only when ready
            httpGet: { path: /health, port: 3000 }
          livenessProbe:               # restart if it hangs
            httpGet: { path: /health, port: 3000 }
            initialDelaySeconds: 10
---
apiVersion: v1
kind: Service
metadata:
  name: nest-api
spec:
  selector: { app: nest-api }          # ← finds pods with this label
  ports:
    - port: 80
      targetPort: 3000
  type: LoadBalancer                   # cloud gives a public IP
```

k8s/mongo.yaml (simplified)

```yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: mongo
spec:
  serviceName: mongo
  replicas: 1
  selector:
    matchLabels: { app: mongo }
  template:
    metadata:
      labels: { app: mongo }
    spec:
      containers:
        - name: mongo
          image: mongo:7
          ports: [{ containerPort: 27017 }]
          volumeMounts:
            - name: mongo-data
              mountPath: /data/db          # ← same idea as theory #52
  volumeClaimTemplates:
    - metadata: { name: mongo-data }
      spec:
        accessModes: ["ReadWriteOnce"]
        resources: { requests: { storage: 5Gi } }
---
apiVersion: v1
kind: Service
metadata:
  name: mongo
spec:
  clusterIP: None                      # headless: stable DNS "mongo-0.mongo"
  selector: { app: mongo }
  ports: [{ port: 27017 }]
```

Secret

```yaml
kubectl create secret generic app-secrets \
  --from-literal=MONGO_URI=mongodb://mongo:27017/schooldb
```

> In production, most teams run the **database outside** Kubernetes (MongoDB Atlas, AWS RDS, DocumentDB) and keep only the stateless apps in K8s. Running databases in K8s works, but it needs more care.

8. kubectl: the commands you'll use

```
kubectl apply -f k8s/                 # create/update everything in folder
kubectl get pods                      # list pods
kubectl get deploy,svc,pvc            # list deployments, services, disks
kubectl describe pod <name>           # details + events (why it failed)
kubectl logs -f <pod-name>            # live logs
kubectl exec -it <pod-name> -- sh     # go inside a container
kubectl port-forward svc/nest-api 3000:80   # access a service locally

kubectl scale deploy nest-api --replicas=10        # scale up
kubectl set image deploy/nest-api nest-api=gourish/nest-api:v2   # new version
kubectl rollout status deploy/nest-api             # watch the rolling update
kubectl rollout undo deploy/nest-api               # rollback ⏪

kubectl delete -f k8s/                # remove everything
```

Try it: self-healing demo

```
kubectl get pods                      # 3 nest-api pods running
kubectl delete pod nest-api-7d9f-abc  # kill one!
kubectl get pods                      # a NEW pod already being created ✅
```

With plain Docker, `docker rm -f` means the container is gone until you start it again.

Rolling update, visualised

```
  v1 v1 v1        start   →  v1 v1 v1 v2     (new pod ready?)
  v1 v1 v2        remove  →  one old pod
  v1 v2 v2        …repeat…
  v2 v2 v2        done. Users never saw downtime 🚀
```

9. Run Kubernetes on your own PC

Tool	How
Docker Desktop	Settings → Kubernetes → Enable. Easiest on Windows ✅
minikube	`minikube start`: a single-node cluster
kind	"Kubernetes IN Docker": a cluster made of Docker containers, popular for CI
k3s	Lightweight Kubernetes for small servers / Raspberry Pi

In production, use a **managed** cluster: **EKS** (AWS), **AKS** (Azure) or **GKE** (Google). The cloud runs the control plane for you.

10. Docker without vs with Kubernetes: full comparison

	Docker / Compose only	Docker + Kubernetes
Servers	1 machine	Many machines (a cluster)
How you work	Commands: "run this"	Desired state: "I want this", and it keeps it true
Container crashes	`restart: always` (same machine only)	Auto-restarts. Moves to another node if the server dies.
Scaling	Manual, 1 machine	`kubectl scale` or automatic (HPA) across nodes
Load balancing	DIY (Nginx)	Built-in (Service, Ingress)
Deploy a new version	Usually some downtime	Rolling update with zero downtime + one-command rollback
Networking	Compose network on one host	Cluster-wide DNS (`mongo`, `nest-api`)
Config / secrets	`.env` files	ConfigMaps / Secrets
Storage	Volumes on that machine	PVCs, cloud disks that follow the pod
Config files	1 short `docker-compose.yml`	Many YAML files (Deployment, Service, Ingress…)
Learning curve	Easy ☺	Steep 😅
Cost	Cheap (1 server)	Higher (cluster + control plane + ops time)
Best for	Dev, learning, small apps, MVPs	Production at scale, microservices, high availability

11. Do I need Kubernetes? 🤔

Use Docker / Compose alone if...	Use Kubernetes if...
It's a single app or a few services	You run many microservices
Traffic is small or predictable	Traffic is large or spiky and needs autoscaling
A few minutes of downtime is OK	You need high availability and zero-downtime deploys
It's a small team with no DevOps person	A team or platform group can manage it
It's a learning, college or MVP project	It's a company product with many users

> **Don't use Kubernetes just because it's popular.** For a small NestJS app, Compose on 1 server, or a simpler managed service (AWS ECS Fargate, Elastic Beanstalk, Render, Railway), is cheaper and easier. Move to K8s when the pain from section 2 actually shows up.

Middle options (between Compose and K8s)

- **Docker Swarm:** built into Docker, very simple clustering (`docker swarm init`), but much less popular today.
- **AWS ECS / Fargate, Google Cloud Run, Azure Container Apps:** you give them a Docker image, and they handle scaling and healing without the full Kubernetes complexity.

12. Typical workflow (Docker + Kubernetes together)

```
1. Write code (NestJS)
       |
2. docker build -t gourish/nest-api:v2 .        ← DOCKER builds the image
       |
3. docker push gourish/nest-api:v2              ← to Docker Hub / AWS ECR
       |
4. Update image tag in k8s/api.yaml
       |
5. kubectl apply -f k8s/                        ← KUBERNETES runs it
       |
6. K8s rolls out v2 across the cluster, heals and scales it
   (usually automated with CI/CD: GitHub Actions, ArgoCD)
```

13. Quick summary

Question	Answer
Docker?	Packages and runs containers (on one machine)
Kubernetes?	Orchestrates many containers on many machines
Who made K8s / when?	Google (inspired by Borg), 2014; now run by the CNCF
Why was it introduced?	Managing hundreds of containers across servers by hand was impossible
Core idea	Desired state + a control loop that keeps fixing reality
Brain	Control plane: API Server, etcd, Scheduler, Controller Manager
Workers	Nodes running kubelet + kube-proxy + container runtime
Main objects	Pod, Deployment, Service, Ingress, ConfigMap, Secret, PVC
Superpowers	Self-healing, scaling, load balancing, rolling updates, rollback
Compare	Compose = one kitchen, Kubernetes = a managed restaurant chain
Rule	Start with Compose. Move to K8s when scale and uptime really need it.